

REPUBLIQUE TUNISIENNE
MINISTERE DE L'ENSEIGNEMENT SUPERIEUR
ET DE LA RECHERCHE SCIENTIFIQUE

UNIVERSITE TUNIS EL MANAR

FACULTE DES SCIENCES DE TUNIS
DEPARTEMENT DES SCIENCES DE L'INFORMATIQUE

RAPPORT DE
PROJET EN NOUVELLES TECHNOLOGIES

PRLens

Systeme automatise de revue de code par intelli-
gence artificielle pour les Pull Requests GitHub

Realise par :

Ismail MECHKENE

Encadre par :

M. Faouzi MOUSSA

Organisme d'accueil : FST – DSI

Annee Universitaire : 2025 – 2026

Fiche de synthese

Dans le cadre de la gestion informatisee des projets et de l'archivage des rapports, les items suivants sont renseignes :

Champ	Valeur
Formation	Licence Appliquee en Sciences de l'Informatique
Annee Universitaire	2025 – 2026
Session	Principale
Auteur	Ismail MECHKENE
Email et telephone du responsable du groupe	[A COMPLETER : email et telephone]
Titre du rapport	PRLens : Systeme automatise de revue de code par intelligence artificielle pour les Pull Requests GitHub
Organisme d'accueil	FST – DSI
Pays d'accueil	Tunisie
Responsable de stage	M. Faouzi MOUSSA
Email du responsable de stage	[A COMPLETER : email du responsable]
Mots-cles	Revue de code, Intelligence artificielle, GitHub, LLM, Automatisation

Charte de non plagiat

Protection de la propriété intellectuelle

Tout travail universitaire doit être réalisé dans le respect intégral de la propriété intellectuelle d'autrui. Pour tout travail personnel, ou collectif, pour lequel le candidat est autorisé à utiliser des documents (textes, images, code source, etc.), celui-ci doit très précisément signaler le crédit (référence complète du texte cité, de la bibliothèque logicielle ou de la source internet utilisée) à la fois dans le corps du texte et dans la bibliographie.

Je soussigné, **Ismail MECHKENE**, étudiant en Licence Appliquée en Sciences de l'Informatique, m'engage à respecter cette charte.

Fait à Tunis, le **[A COMPLETER : date]**

Signature : _____

Remerciements

Je tiens tout d'abord à exprimer ma sincère gratitude à mon encadrant, **M. Faouzi MOUSSA**, pour sa disponibilité, la pertinence de ses conseils et la rigueur qu'il a su transmettre tout au long de ce projet. Ses orientations méthodologiques ont été déterminantes dans la structuration de la démarche adoptée, depuis l'analyse du besoin jusqu'à la validation de la solution.

Mes remerciements s'adressent également à l'ensemble du corps enseignant du Département des Sciences de l'Informatique de la Faculté des Sciences de Tunis, pour la qualité de la formation dispensée durant ce cursus, qui a constitué le socle des compétences mobilisées dans ce projet.

Je remercie enfin toute personne ayant, de près ou de loin, contribué à l'aboutissement de ce travail.

Table des matières

Fiche de synthese	i
Charte de non plagiat	ii
Remerciements	iii
Liste des figures	vii
Liste des tableaux	viii
Liste des annexes	ix
Introduction generale	1
1 Presentation du cadre du projet et de l'encadrant	3
1.1 Cadre general du projet	3
1.2 Encadrant	3
1.3 Periode du projet	3
Conclusion du chapitre	4
2 Etat de l'art et fondements metiers	5
2.1 Contexte metier du projet	5
2.1.1 Presentation du domaine d'application	5
2.1.2 Analyse des besoins et des problematiques specifiques	5
2.2 Problematique metier	5
2.2.1 Identification des principaux defis	6
2.2.2 Objectifs vises par le projet	6
2.3 Proposition de valeur	6
2.3.1 Valeur ajoutee pour les parties prenantes	6
2.3.2 Avantages et benefices attendus	7
2.4 Etude de marche	7
2.4.1 Analyse de l'environnement concurrentiel	7
2.4.2 Opportunités et menaces potentielles	8
2.5 Backlog metier	8
2.5.1 Fonctionnalites et exigences metier	8
Conclusion du chapitre	9

3	Fondements technologiques et cadre conceptuel du projet	10
3.1	Revue de la littérature	10
3.1.1	Synthese des travaux existants	10
3.1.2	Principales avancees mobilisees	10
3.2	Contexte informatique	10
3.2.1	Concepts fondamentaux	10
3.2.2	Langages et technologies retenus	11
3.3	Outils et methodologies	11
3.3.1	Environnements de developpement	11
3.3.2	Patrons de conception mobilises	12
3.3.3	Organisation du travail	12
3.4	Cadre conceptuel	12
	Conclusion du chapitre	12
4	Conception de la solution informatique	13
4.1	Introduction a la conception	13
4.2	Analyse des besoins	13
4.2.1	Synthese des besoins	13
4.2.2	Priorisation	13
4.3	Architecture globale du systeme	13
4.3.1	Vue d'ensemble	14
4.3.2	Composants principaux	14
4.4	Modelisation des donnees	15
4.4.1	Schema de la base de donnees	15
4.4.2	Contraintes de suppression	15
4.5	Interfaces utilisateur	16
4.5.1	Structure du tableau de bord	16
4.5.2	Parcours d'utilisation type	16
4.5.3	Choix d'ergonomie	17
4.6	Algorithmes et methodes	17
4.6.1	Analyse parallele des fichiers	17
4.6.2	Calcul du score et du verdict	18
4.6.3	Gestion des echecs partiels	18
4.7	Securite et confidentialite	18
4.7.1	Strategies de securite mises en place	18
4.7.2	Protection des donnees sensibles	19
4.8	Scenarios de cas d'utilisation	19
4.9	Plan de developpement	19
	Conclusion du chapitre	20

5	Realisation et validation	21
5.1	Introduction a la realisation	21
5.2	Environnement de developpement	21
5.2.1	Outils, plateformes et langages	21
5.2.2	Justification des choix technologiques	21
5.3	Developpement des modules	21
5.3.1	Le moteur de revue	22
5.3.2	Le tableau de bord	22
5.4	Tests et debogage	22
5.4.1	Strategie de tests	22
5.4.2	Resultats obtenus	22
5.4.3	Processus de debogage	23
5.5	Integration des composants	23
5.6	Validation et verification	24
5.6.1	Evaluation par rapport aux besoins initiaux	24
5.6.2	Conformite aux specifications	24
5.7	Optimisation des performances	24
5.8	Documentation technique	25
5.9	Formation et transfert de competences	25
	Conclusion du chapitre	25
	Conclusion generale et perspectives	26
	Bibliographie	28
	Appendices	29
A	Exemple de fichier de configuration <code>.aireviewer.yml</code>	30
B	Exemple de commentaire genere par PRLens	31
C	Extrait du modele de donnees (SQLAlchemy)	32
D	Liste complete des variables d'environnement	33
E	Liste complete des points d'accès de l'API REST	34
	Resume	35
	Abstract	36

Table des figures

4.1	Architecture globale du pipeline de revue PRLens	14
4.2	Modele de donnees relationnel (extrait des modeles SQLAlchemy) . . .	15
4.3	Diagramme de sequence — connexion, configuration et revue d'une Pull Request	17

Liste des tableaux

2.1	Positionnement de PRLens par rapport aux categories d'outils existantes	8
2.2	Backlog metier synthetique du projet PRLens	9
3.1	Pile technologique du projet PRLens	11
4.1	Regles de decision du verdict de revue	18
4.2	Principaux cas d'utilisation	19
5.1	Couverture de tests du moteur de revue par module	23
5.2	Resultats du harnais d'evaluation (derniere execution)	24
D.1	Variables d'environnement du serveur PRLens	33
E.1	Points d'accès exposes par le serveur FastAPI	34

Liste des annexes

- **Annexe A** — Exemple de fichier de configuration `.aireviewer.yml`
- **Annexe B** — Exemple de commentaire genere par PRLens
- **Annexe C** — Modele de donnees (extrait SQLAlchemy)
- **Annexe D** — Liste complete des variables d’environnement
- **Annexe E** — Liste complete des points d’acces de l’API REST

Introduction generale

La revue de code (*code review*) est aujourd’hui une etape incontournable du cycle de developpement logiciel : elle conditionne la qualite, la securite et la maintenabilite d’une base de code, et constitue le principal garde-fou avant l’integration de tout changement dans une branche principale. Sur la plateforme GitHub, ce processus prend la forme de la *Pull Request* (PR), un mecanisme qui propose un ensemble de modifications, ouvre une discussion et exige generalement l’approbation d’un ou plusieurs relecteurs humains avant fusion.

Or cette relecture humaine se heurte a des limites bien connues : elle est chronophage, elle depend fortement de la disponibilite et de l’attention du relecteur, et elle est sujette a l’inconstance — deux relecteurs, voire un meme relecteur a deux moments differents, ne detectent pas systematiquement les memes problemes. Dans des equipes ou le volume de Pull Requests est important, ce goulot d’etranglement ralentit le rythme de livraison et laisse regulierement passer des defauts evitables (secrets codes en dur, injections, fonctions trop complexes, absence de gestion d’erreurs, etc.).

C’est dans ce contexte qu’a ete concu **PRLens**, l’objet du present rapport : un systeme de revue de code automatise qui s’appuie sur un grand modele de langage (*Large Language Model*, LLM) pour analyser le contenu exact d’une Pull Request et produire, sans intervention humaine, des commentaires en ligne, un score de qualite chiffre, un verdict de revue formel (Approuver / Demander des changements / Commenter) et des etiquettes de classification. PRLens ne remplace pas la revue humaine : il agit comme un premier filtre systematique, rapide et reproductible, qui traite les problemes les plus frequents et laisse au relecteur humain le soin de juger des aspects fonctionnels et architecturaux de plus haut niveau.

Le projet se decompose en deux volets complementaires. Le premier est le **moteur de revue** lui-meme, deployable de deux manieres : comme etape d’un pipeline GitHub Actions (sans serveur a heberger), ou comme serveur *webhook* permanent installe une fois pour tous les depots d’une organisation. Le second volet est un **tableau de bord web** (*dashboard*) qui permet a un utilisateur de se connecter avec son compte GitHub, de connecter ses depots en un clic, de configurer le comportement de la revue par depot, et de consulter l’historique et les tendances de qualite mesurees au fil du temps.

Ce rapport est structure en quatre chapitres, precedes d’une presentation du cadre du projet. Le premier chapitre situe le projet dans son contexte metier, degage la problematique et la proposition de valeur, et positionne PRLens par rapport aux solutions existantes sur le marche. Le deuxieme chapitre presente les fondements technologiques

mobilises — langages, frameworks, patrons de conception — ainsi que le cadre conceptuel retenu. Le troisieme chapitre detaille la conception de la solution : architecture globale, modelisation des donnees, interfaces, algorithmes et securite. Le quatrieme chapitre decrit la realisation effective du systeme et sa validation, a travers les tests unitaires et un protocole d’evaluation quantitatif de la pertinence des revues produites. Le rapport se cloture par une conclusion generale qui dresse le bilan du travail accompli et ouvre sur des perspectives d’evolution.

CHAPITRE 1

Presentation du cadre du projet et de l'encadrant

1.1 Cadre general du projet

Le present projet a ete realise dans le cadre du cursus de Licence Appliquee en Sciences de l'Informatique de la Faculte des Sciences de Tunis (FST), au sein du Departement des Sciences de l'Informatique. Il s'inscrit dans la categorie des projets a caractere applicatif portant sur les nouvelles technologies, avec pour terrain d'application concret l'automatisation d'un processus de developpement logiciel a l'aide de l'intelligence artificielle generative.

L'organisme d'accueil du projet est la **Division/Direction des Systemes d'Information (DSI)** de la FST, qui a fourni le cadre d'encadrement pedagogique et technique necessaire au bon deroulement du travail.

1.2 Encadrant

Le projet a ete encadre par **M. Faouzi MOUSSA**, enseignant au sein du Departement des Sciences de l'Informatique de la FST. L'encadrement a porte a la fois sur la methodologie de conduite de projet (structuration du travail, jalons, livrables) et sur les choix techniques structurants du systeme, notamment l'architecture logicielle, l'integration avec les services externes (GitHub, Azure OpenAI) et les exigences de qualite (couverture de tests, evaluation de la pertinence des sorties du modele).

1.3 Periode du projet

Le developpement du projet s'est deroule durant l'annee universitaire 2025–2026.
[A COMPLETER : dates precises de debut et de fin du projet]

Conclusion du chapitre

Ce chapitre a situe le cadre institutionnel et humain dans lequel le projet PRLens a ete mene. Le chapitre suivant entre dans le vif du sujet en presentant le contexte metier de la revue de code automatisee, la problematique qui en decoule et la proposition de valeur de PRLens.

CHAPITRE 2

Etat de l'art et fondements metiers

2.1 Contexte metier du projet

2.1.1 Presentation du domaine d'application

Le domaine d'application du projet est celui de l'**assurance qualite logicielle**, et plus precisement celui de la **revue de code assistee par intelligence artificielle** dans un flux de travail base sur Git et GitHub. Une Pull Request materialise une proposition de changement : elle regroupe un ensemble de *commits*, un diff (les lignes ajoutees, supprimees ou modifiees) et une conversation associee. La revue de cette PR est le moment ou l'equipe verifie que le changement propose est correct, sur, performant et coherent avec le reste du code avant de l'integrer.

2.1.2 Analyse des besoins et des problematiques specifiques

L'analyse du terrain met en evidence plusieurs besoins recurrents chez les equipes de developpement :

- **Rapidite de retour** — un developpeur qui attend plusieurs heures, voire plusieurs jours, une premiere relecture perd le contexte mental de son changement et voit sa productivite baisser.
- **Systematicite** — certaines classes de problemes (secrets codes en dur, injections SQL, fonctions demesurement complexes) doivent etre detectees a chaque fois, independamment de la charge de travail ou de la fatigue du relecteur humain.
- **Tracabilite** — l'equipe a besoin d'un historique exploitable : quel depot est le plus souvent en cause, quel type de probleme revient, la qualite du code progresse-t-elle dans le temps ?
- **Faible cout d'adoption** — une solution qui exige une reconfiguration lourde de l'infrastructure existante peine a etre adoptee par des equipes deja tres sollicitees.

2.2 Problematique metier

2.2.1 Identification des principaux defis

La problematique centrale du projet peut se formuler ainsi : *comment fournir, de maniere automatique et systematique, une premiere revue de qualite professionnelle sur chaque Pull Request GitHub, sans imposer de contrainte d'infrastructure lourde et en laissant a l'equipe le controle fin du comportement de cette revue ?*

Cette problematique se decompose en plusieurs defis techniques et metiers :

1. Analyser un diff de maniere suffisamment fine pour produire des commentaires *sur la ligne exacte concernee*, et non une observation generale peu actionnable.
2. Produire un verdict et un score reproductibles a partir d'une source par nature non deterministe (un LLM), afin que le resultat reste exploitable en pratique (etiquetage, blocage de fusion, etc.).
3. Permettre a chaque equipe de personnaliser le comportement de la revue (langages cibles, fichiers exclus, seuils de score, relecteurs a affecter) sans reecrire de code.
4. S'integrer nativement dans deux contextes d'usage distincts : celui d'une equipe qui prefere un pipeline sans etat (*GitHub Actions*), et celui d'une equipe qui prefere une installation centralisee, sans configuration par depot (*application GitHub*).

2.2.2 Objectifs vises par le projet

Les objectifs retenus pour le projet sont les suivants :

- Concevoir un moteur de revue capable de detecter des problemes de **securite**, de **qualite**, de **performance** et de **documentation** dans un diff de Pull Request.
- Produire pour chaque revue un **score chiffre** (0 a 100) et un **verdict formel** exploitable directement par GitHub (Approve / Request changes / Comment).
- Offrir deux modes de deploiement — *GitHub Actions* et *webhook applicatif* — partageant le meme moteur de revue.
- Fournir un **tableau de bord web** permettant la connexion des depots, la configuration par depot et la consultation de l'historique des revues, sans passer par un fichier de configuration.
- Valider objectivement la pertinence des revues produites au moyen d'un protocole d'evaluation quantitatif, et non uniquement par des tests unitaires.

2.3 Proposition de valeur

2.3.1 Valeur ajoutee pour les parties prenantes

PRLens apporte une valeur differenciee a trois categories d'acteurs :

- **Le developpeur** recoit un retour en quelques dizaines de secondes apres l'ouverture ou la mise a jour de sa Pull Request, avec des suggestions de correction concretes plutot qu'une simple liste de reproches.
- **Le responsable technique / lead** dispose d'un tableau de bord centralise donnant une vue d'ensemble de la sante du code sur l'ensemble des depots geres, avec un historique exploitable pour identifier les tendances.
- **L'organisation** reduit le temps humain consacre a la relecture des problemes les plus frequents et documentes, et gagne en systematicite sur des categories de risques sensibles comme la securite.

2.3.2 Avantages et benefices attendus

- Reduction du delai moyen entre l'ouverture d'une Pull Request et son premier retour.
- Detection systematique d'une classe de vulnerabilites connues, alignee sur le referentiel *OWASP Top 10*.
- Zero configuration necessaire pour une equipe qui installe l'application GitHub : chaque nouveau depot rattache est revu automatiquement.
- Personnalisation fine du comportement, par depot, via le tableau de bord ou via un fichier de configuration versionne dans le depot lui-meme.

2.4 Etude de marche

2.4.1 Analyse de l'environnement concurrentiel

Le marche des outils de revue de code assistee par intelligence artificielle a connu une expansion rapide, portee par la generalisation des LLM. Le tableau 2.1 positionne PRLens par rapport a quelques categories d'outils representatives du marche.

Categorie	Exemples de solutions	Positionnement de PRLens
Analyse statique classique	SonarQube, ESLint, Pylint	PRLens complete l'analyse statique par une comprehension semantique du changement, au-dela des regles syntaxiques fixes.
Assistants de revue bases sur un LLM (SaaS)	CodeRabbit, GitHub Copilot (revue de PR)	PRLens propose une architecture ouverte, auto-hebergeable, avec deux modes de deploiement et un modele de scoring/verdict transparent et configurable.
Scanners de securite dedies	Snyk Code, Semgrep	PRLens integre la securite comme une des quatre dimensions analysees (avec qualite, performance, documentation), au sein d'un seul flux de revue unifie.

Tableau 2.1 — Positionnement de PRLens par rapport aux categories d'outils existantes

2.4.2 Opportunités et menaces potentielles

Opportunités — l'adoption massive des LLM de nouvelle generation (GPT-4o et successeurs) rend desormais accessible, a cout raisonnable, une comprehension fine du code source qui etait auparavant reservee a des systemes bases sur des regles rigides. L'écosysteme GitHub (Actions, GitHub Apps, API REST) fournit par ailleurs des points d'integration matures et bien documentes.

Menaces — la dependance a un fournisseur de LLM externe (ici Azure OpenAI) introduit un risque de disponibilite et de cout variable. Le caractere non deterministe des LLM impose egalement une vigilance constante sur le taux de faux positifs, sous peine de degrader la confiance des utilisateurs dans l'outil.

2.5 Backlog metier

2.5.1 Fonctionnalités et exigences metier

Le backlog metier du projet a ete organise autour de deux grands ensembles fonctionnels : le **moteur de revue** et le **tableau de bord**. Le tableau 2.2 en presente une synthese priorisee.

Priorite	Fonctionnalite	Module
Haute	Analyser un diff de PR et produire des commentaires en ligne classes par severite	Moteur
Haute	Calculer un score de qualite et un verdict de revue formel	Moteur
Haute	Authentifier l'application aupres de GitHub (App ou jeton personnel)	Moteur
Haute	Executer la revue depuis un pipeline GitHub Actions	Moteur
Haute	Executer la revue depuis un serveur webhook permanent	Moteur
Moyenne	Permettre la connexion des utilisateurs par OAuth GitHub	Dashboard
Moyenne	Connecter / deconnecter un depot depuis l'interface	Dashboard
Moyenne	Configurer par depot les seuils, langages et fichiers exclus	Dashboard
Moyenne	Afficher l'historique des revues et les tendances de score	Dashboard
Basse	Suspendre temporairement la revue d'un depot (bascule actif/inactif)	Dashboard
Basse	Affecter automatiquement des relecteurs selon le type de probleme	Moteur

Tableau 2.2 – Backlog metier synthetique du projet PRLens

Conclusion du chapitre

Ce chapitre a etabli le contexte metier du projet, la problematique qu'il adresse et la valeur qu'il apporte, en le situant par rapport aux solutions existantes sur le marche. Le chapitre suivant presente le socle technologique et conceptuel retenu pour repondre a ce cahier des charges.

CHAPITRE 3

Fondements technologiques et cadre conceptuel du projet

3.1 Revue de la littérature

3.1.1 Synthèse des travaux existants

L’usage de modèles de langage pour l’analyse de code source s’appuie sur des travaux de recherche en traitement automatique du langage appliqué au code (*code intelligence*), dont les modèles de la famille GPT constituent une des réalisations les plus abouties à ce jour. Ces modèles, entraînés sur de très larges corpus incluant du code source, sont capables d’inférer l’intention d’un extrait de code, d’identifier des anomalies et de proposer des corrections en langage naturel — une capacité directement exploitée par PRLens.

3.1.2 Principales avancées mobilisées

Deux avancées techniques récentes rendent ce projet possible dans des conditions de coût et de fiabilité raisonnables :

- Le **mode de sortie structurée** (JSON strict) proposé par les API de LLM modernes, qui permet de contraindre le modèle à répondre selon un schéma exploitable programmatically, plutôt que sous forme de texte libre à analyser.
- Les **applications GitHub** (*GitHub Apps*), qui offrent un modèle d’authentification et d’autorisation granulaire (jeton d’installation à durée de vie courte, permissions scopees par dépôt) mieux adapté à un usage machine-à-machine qu’un jeton personnel classique.

3.2 Contexte informatique

3.2.1 Concepts fondamentaux

Le projet repose sur trois familles de concepts informatiques : (i) l’**intégration continue** et les événements Git (PR ouverte, mise à jour, fermeture), (ii) les **API**

REST et l'authentification par jeton pour dialoguer avec GitHub et avec le fournisseur de LLM, et (iii) l'**architecture web classique client/serveur** pour le tableau de bord, avec une authentification federee (OAuth 2.0).

3.2.2 Langages et technologies retenus

Le tableau 3.1 recapitule la pile technologique du projet.

Couche	Technologie	Role
Langage — moteur	Python 3.11	Implementation du pipeline de revue
Framework serveur	FastAPI	Serveur webhook et API REST du tableau de bord
Client GitHub	PyGithub	Acces a l'API REST de GitHub (PR, commentaires, labels)
Client LLM	OpenAI SDK (Azure OpenAI)	Appels au modele <code>gpt-4o</code> avec sortie JSON structuree
Validation de donnees	Pydantic v2	Modelisation stricte des entites (PR, commentaire, resultat)
Persistence	PostgreSQL + SQLAlchemy (ORM)	Stockage des utilisateurs, installations et revues
Serveur ASGI	Uvicorn	Execution du serveur FastAPI
Langage — interface	TypeScript	Typage statique du tableau de bord
Framework interface	React 19 + Vite	Interface utilisateur du tableau de bord (SPA)
Routage cote client	React Router	Navigation entre les pages du tableau de bord
Authentification	OAuth 2.0 (GitHub) + JWT	Connexion utilisateur et securisation de l'API
Conteneurisation	Docker	Empaquetage de l'action GitHub Actions

Tableau 3.1 – Pile technologique du projet PRLens

3.3 Outils et methodologies

3.3.1 Environnements de developpement

Le developpement a ete mene sous un environnement de developpement integre (EDI) supportant nativement Python et TypeScript, avec integration Git. Le depot est heberge sur GitHub, qui sert egalement de cible d'integration fonctionnelle du produit lui-meme (PRLens revoit ses propres Pull Requests en conditions reelles).

3.3.2 Patrons de conception mobilises

Plusieurs patrons de conception structurent le code du moteur de revue :

- **Facade** — la classe **Agent** expose une methode unique (**run**) qui masque l’orchestration interne (recuperation de la PR, analyse, publication) aux deux points d’entree (mode Actions et mode webhook).
- **Strategy** — l’authentification GitHub (jeton personnel ou application GitHub) est selectionnee dynamiquement selon les variables d’environnement presentes, sans modifier le code appelant.
- **Fan-out / Worker pool** — l’analyseur distribue un appel au LLM par fichier modifie sur un **ThreadPoolExecutor**, paralleisant l’analyse tout en respectant une limite configurable de travailleurs simultanes (**max_workers**).
- **Repository implicite** — la couche SQLAlchemy isole le reste de l’application du detail du schema relationnel, via des modeles declaratifs (**User**, **Installation**, **Review**, **ReviewComment**).

3.3.3 Organisation du travail

Le developpement a ete mene de maniere iterative, chaque increment portant sur un module fonctionnel verifiable independamment (authentification GitHub, analyseur, publication, puis tableau de bord), avec executions regulieres de la suite de tests automatises comme filet de securite.

3.4 Cadre conceptuel

Le cadre conceptuel retenu pour le projet repose sur une separation nette entre trois responsabilites : **recuperer** l’etat d’une Pull Request depuis GitHub, **analyser** ce contenu a l’aide d’un LLM pour produire des constats structures, et **publier** le resultat de cette analyse sous forme exploitable par l’humain (commentaires, score, verdict, etiquettes). Cette separation, detaillee au chapitre suivant, permet de reutiliser integralement le meme moteur d’analyse dans les deux modes de deploiement du projet.

Conclusion du chapitre

Ce chapitre a presente le socle technologique et les patrons de conception qui structurent PRLens. Le chapitre suivant detaille la conception de la solution a partir de ce cadre : architecture globale, modelisation des donnees et interfaces.

CHAPITRE 4

Conception de la solution informatique

4.1 Introduction a la conception

Ce chapitre presente la conception detaillee de PRLens, en partant des besoins degages au chapitre 1 et du cadre technologique pose au chapitre 2. L'objectif de la phase de conception a ete de definir une architecture capable de servir un meme pipeline de revue depuis deux points d'entree distincts, tout en isolant proprement la couche de persistance necessaire au tableau de bord.

4.2 Analyse des besoins

4.2.1 Synthese des besoins

Les besoins fonctionnels retenus pour la conception sont les suivants : analyser un diff de Pull Request par langage et par fichier ; calculer un score et un verdict deterministes a partir de constats produits par un LLM ; publier des commentaires en ligne, un resume, des etiquettes et un verdict formel sur GitHub ; permettre la connexion et la configuration des depots via une interface web ; et journaliser chaque revue effectuee pour restitution ulterieure.

4.2.2 Priorisation

La priorisation reprend le backlog du tableau 2.2 : le moteur de revue (recuperation, analyse, publication) constitue le socle indispensable, le tableau de bord et ses fonctionnalites de configuration/historique en sont des extensions a valeur ajoutee.

4.3 Architecture globale du systeme

4.3.1 Vue d'ensemble

PRLens partage un unique pipeline de revue (la classe **Agent**) derriere deux points d'entree. En mode *Actions*, `main.py` l'execute une fois par evenement de Pull Request, a l'interieur d'un conteneur Docker. En mode *webhook*, un serveur FastAPI invoque ce meme pipeline a chaque evenement reçu, et persiste en plus le resultat pour le tableau de bord. La figure 4.1 illustre ce flux.

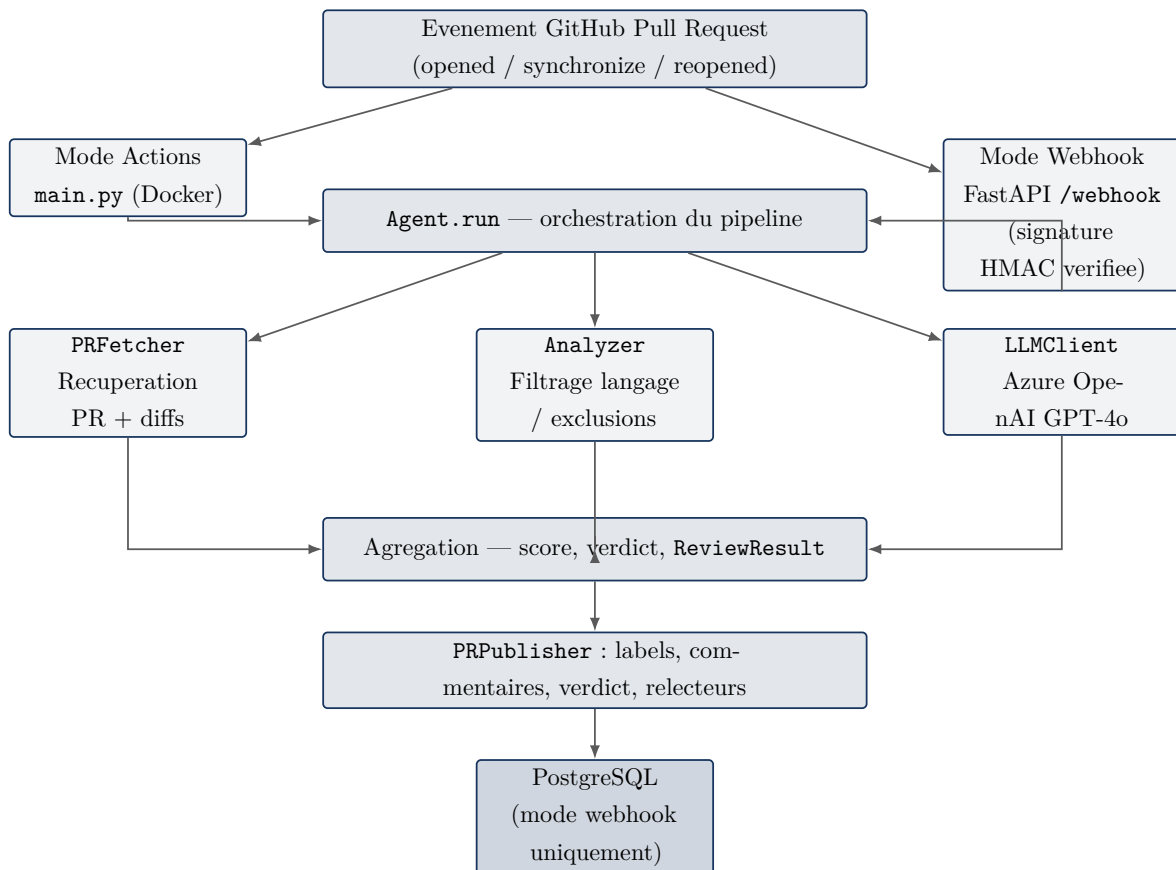


Figure 4.1 – Architecture globale du pipeline de revue PRLens

4.3.2 Composants principaux

- **PRFetcher** — recupere la Pull Request et ses fichiers modifies via PyGithub, et les convertit en modeles internes (`PR`, `FileChange`).
- **Analyzer** — filtre les fichiers par langage cible et par motif d'exclusion, puis distribue un appel LLM par fichier restant sur un `ThreadPoolExecutor`.
- **LLMClient** — encapsule l'appel a l'API Azure OpenAI et impose un contrat de sortie JSON strict (type de probleme, severite, ligne, message, suggestion).
- **Agregation** — calcule le score global a partir des severites remonteés et determine le verdict (voir §4.6.2).
- **PRPublisher** — traduit le resultat agrege en actions GitHub : etiquettes, commen-

taire de synthese, commentaires en ligne, soumission de la revue, affectation de relecteurs.

- **Serveur FastAPI** (`prlens/webhook/app.py`) — en mode webhook, recoit et verifie les evenements, invoque l’agent en tache de fond, et expose en plus l’API REST consommee par le tableau de bord.

4.4 Modelisation des donnees

4.4.1 Schema de la base de donnees

La persistance necessaire au tableau de bord repose sur quatre entites principales : **User**, **Installation**, **Review** et **ReviewComment**. La figure 4.2 en presente le schema relationnel.

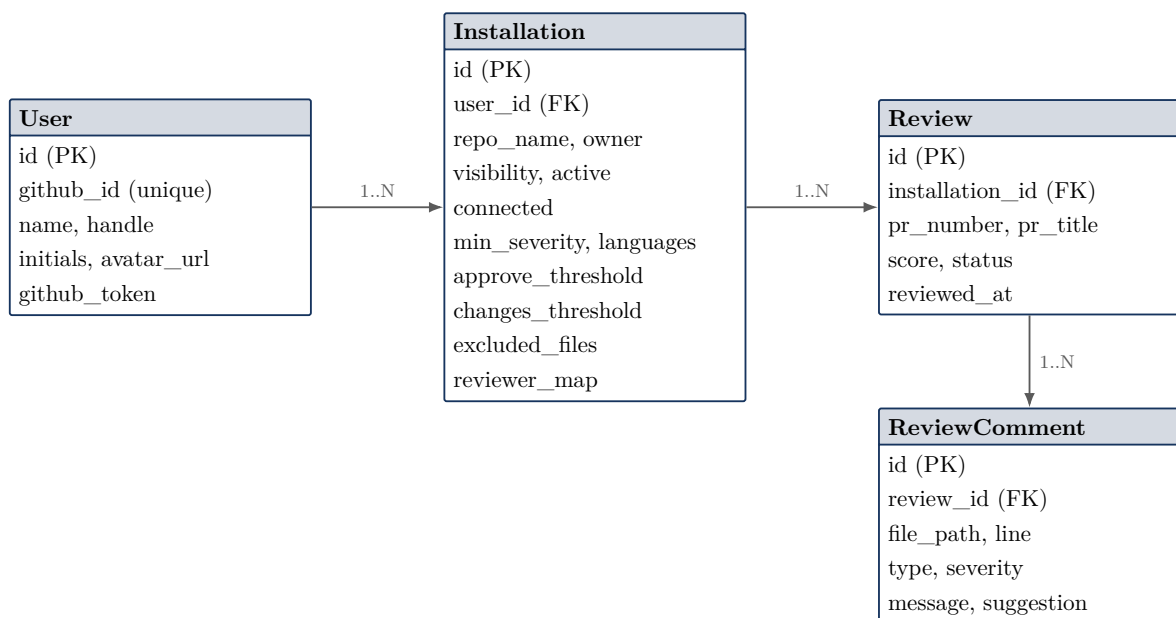


Figure 4.2 – Modele de donnees relationnel (extrait des modeles SQLAlchemy)

Un utilisateur (**User**) possede plusieurs installations (**Installation**), chacune representant le rattachement d’un depot GitHub a un compte, avec ses parametres de revue propres. Chaque installation accumule un historique de revues (**Review**), et chaque revue detaille ses constats individuels (**ReviewComment**). La contrainte d’unicite portant sur le couple (`user_id`, `repo_name`) garantit qu’un meme utilisateur ne connecte pas deux fois le meme depot.

4.4.2 Contraintes de suppression

Les relations sont declarees avec `cascade="all, delete-orphan"` : la suppression d’un utilisateur entraine celle de ses installations, et la suppression (deconnexion)

d'une installation entraine celle de son historique de revues, garantissant l'absence d'enregistrements orphelins.

4.5 Interfaces utilisateur

4.5.1 Structure du tableau de bord

Le tableau de bord est une application web monopage (*SPA*) React organisee autour d'une barre laterale de navigation persistante et d'une zone de contenu principale. Les pages principales sont : la page d'accueil (*landing page*), le tableau de bord (statistiques, tendances), la page de connexion de depots, la page de detail d'un depot (parametres, historique) et les pages statiques (documentation, mentions legales).

4.5.2 Parcours d'utilisation type

La figure 4.3 illustre le parcours complet d'un utilisateur, de la connexion jusqu'a la reception d'une revue automatique sur une Pull Request qu'il a ouverte sur un depot connecte.

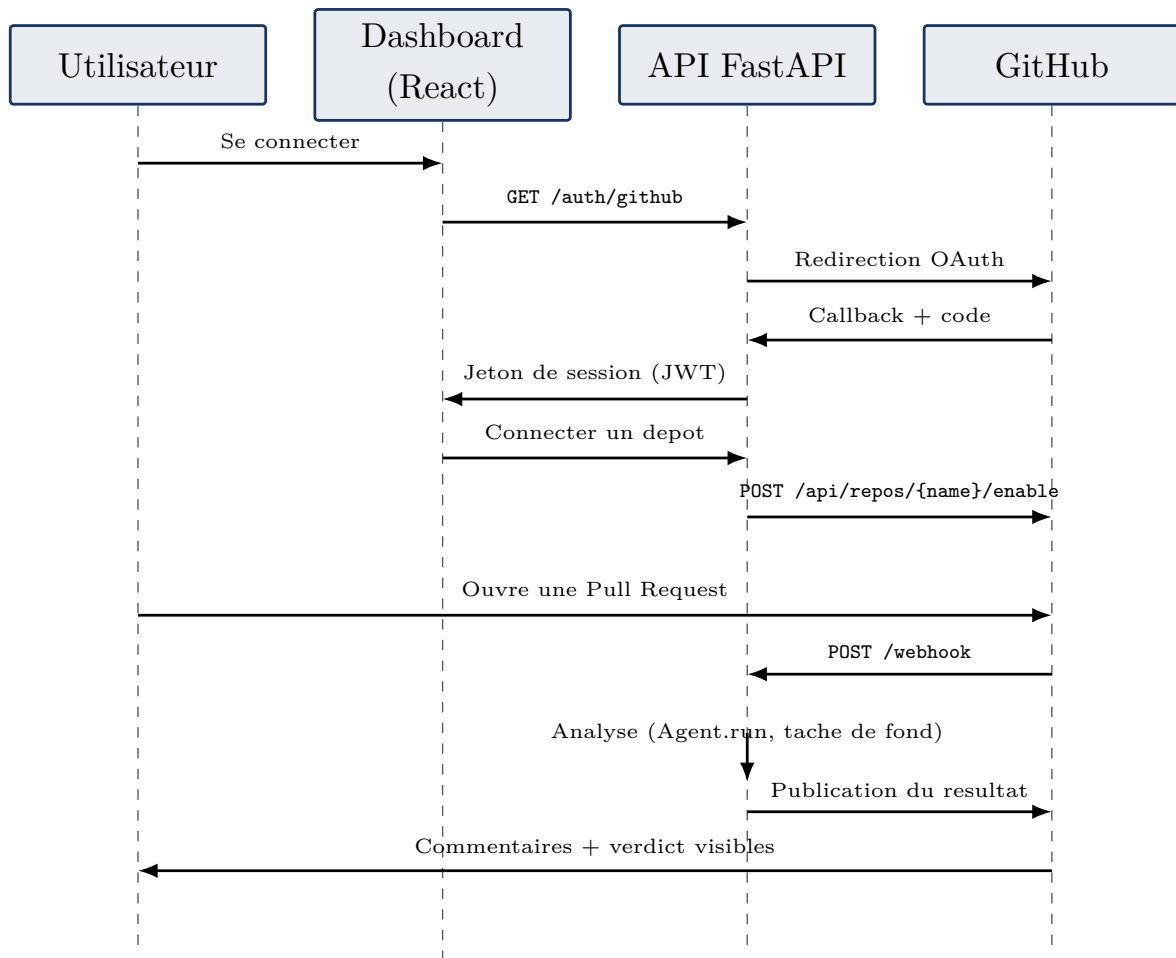


Figure 4.3 – Diagramme de séquence — connexion, configuration et revue d'une Pull Request

4.5.3 Choix d'ergonomie

L'authentification s'appuie exclusivement sur GitHub (OAuth), évitant à l'utilisateur la création d'un compte et d'un mot de passe supplémentaires. La bascule *actif* / *inactif* par dépôt permet de suspendre les revues sans perdre l'historique déjà accumulé, un choix ergonomique motivé par le besoin d'essais temporaires (par exemple pendant une migration de code volumineuse).

4.6 Algorithmes et méthodes

4.6.1 Analyse parallèle des fichiers

Pour chaque fichier retenu par le filtre de langage/exclusion, l'analyseur soumet un appel indépendant au LLM. Ces appels sont distribués sur un `ThreadPoolExecutor` dont la taille est bornée par le paramètre `max_workers` (5 par défaut), ce qui borne le nombre de requêtes simultanées envoyées au fournisseur de LLM tout en parallélisant

l’analyse d’une Pull Request touchant plusieurs fichiers.

4.6.2 Calcul du score et du verdict

Le score de qualite est calcule sur une echelle de 0 a 100, par penalites cumulees selon la severite de chaque constat remonte : -25 points par probleme critique, -10 par erreur, -4 par avertissement et -1 par information. Le verdict final est ensuite derive de ce score et de la presence ou non de problemes critiques, selon la logique presentee au tableau 4.1.

Condition	Verdict	Parametre
Score strictement superieur au seuil ET aucun probleme critique	Approve	<code>approve_threshold</code> (80 par default)
Score inferieur au seuil OU au moins un probleme critique	Request changes	<code>changes_threshold</code> (50 par default)
Cas intermediaire	Comment	—

Tableau 4.1 – Regles de decision du verdict de revue

4.6.3 Gestion des echecs partiels

Lorsque l’analyse d’un ou plusieurs fichiers echoue (erreur reseau, reponse LLM non conforme au schema attendu), PRLens distingue un etat **Incomplete** (une partie des fichiers a pu etre analysee) d’un etat **Total failure** (aucun fichier n’a pu etre analyse), plutot que de faire echouer silencieusement l’ensemble de la revue.

4.7 Securite et confidentialite

4.7.1 Strategies de securite mises en place

- **Verification de signature webhook** — chaque requete recue sur `/webhook` est verifiee par comparaison HMAC-SHA256 de l’en-tete `X-Hub-Signature-256` avec le secret partage `GITHUB_WEBHOOK_SECRET`; toute requete non conforme est rejetee.
- **Jetons a duree de vie courte** — l’authentification en tant qu’application GitHub repose sur un JWT signe avec la cle privee de l’application, echange contre un jeton d’installation renouvele automatiquement avant expiration.
- **Session utilisateur sans cookie inter-domaine** — l’API et le tableau de bord pouvant etre servis sur des domaines distincts, l’authentification utilisateur repose sur un JWT de session transmis en en-tete `Authorization: Bearer`, et non sur un cookie.

- **Cle de signature obligatoire en production** — `SESSION_SECRET` doit être définie et identique sur toutes les instances du serveur ; en son absence, une cle aléatoire par processus est utilisée et un avertissement est journalisé au démarrage.

4.7.2 Protection des données sensibles

La cle privée de l'application GitHub est manipulée sous forme encodée en base64 dans les environnements conteneurisés (les fichiers `.pem` étant exclus du dépôt et de l'image Docker), et jamais journalisée en clair. Les jetons GitHub des utilisateurs sont conservés côté serveur uniquement, jamais transmis au client.

4.8 Scenarios de cas d'utilisation

Le tableau 4.2 synthétise les principaux cas d'utilisation identifiés pour les deux acteurs du système.

Acteur	Cas d'utilisation
Developpeur	Ouvrir / mettre à jour une Pull Request et recevoir automatiquement une revue
Utilisateur du tableau de bord	Se connecter avec GitHub
Utilisateur du tableau de bord	Connecter / déconnecter un dépôt
Utilisateur du tableau de bord	Configurer les paramètres de revue d'un dépôt
Utilisateur du tableau de bord	Activer / désactiver la revue d'un dépôt
Utilisateur du tableau de bord	Consulter l'historique et les tendances de score
Administrateur GitHub	Installer l'application GitHub sur une organisation

Tableau 4.2 – Principaux cas d'utilisation

4.9 Plan de développement

Le développement a été découpé en quatre itérations fonctionnelles : (1) moteur de revue et authentification GitHub, valides par la suite de tests unitaires ; (2) mode webhook et persistance des revues ; (3) tableau de bord — authentification OAuth et gestion des dépôts ; (4) tableau de bord — historique, statistiques et paramètres avancés. Chaque itération s'est clôturée par une exécution complète de la suite de

tests et, pour les iterations touchant au moteur de revue, par une passe du harnais d'évaluation (chapitre 4).

Conclusion du chapitre

Ce chapitre a detaille la conception retenue pour PRLens : une architecture partagee entre deux points d'entree, un modele de donnees relationnel simple mais suffisant, et des mecanismes de securite adaptes a un systeme qui dialogue avec des services tiers sensibles. Le chapitre suivant decrit la realisation effective de cette conception et sa validation.

CHAPITRE 5

Realisation et validation

5.1 Introduction a la realisation

Ce chapitre presente la mise en œuvre concrete de la conception decrite au chapitre 3, ainsi que le protocole de validation retenu pour s'assurer que le systeme repond effectivement au besoin. La validation de PRLens repose sur deux piliers complementaires : une suite de tests unitaires classique portant sur la logique deterministe du systeme, et un harnais d'evaluation dedie mesurant la pertinence des revues produites par le LLM, dont le comportement est par nature non deterministe.

5.2 Environnement de developpement

5.2.1 Outils, plateformes et langages

Le moteur de revue est developpe en Python 3.11 et expose par un serveur FastAPI execute par Uvicorn. Le tableau de bord est developpe en TypeScript avec React 19, construit par Vite. La persistance repose sur PostgreSQL, accede via l'ORM SQLAlchemy. Le mode Actions est empaquete sous forme d'image Docker declaree dans `action.yml`.

5.2.2 Justification des choix technologiques

FastAPI a ete retenu pour sa prise en charge native de la validation de donnees par Pydantic (reutilisant directement les memes modeles que le moteur de revue) et sa capacite a executer des taches de fond (`BackgroundTasks`) sans bloquer la reponse HTTP immediate attendue par GitHub. React et TypeScript ont ete choisis pour la robustesse du typage statique dans une interface manipulant des donnees structurees issues de l'API. PostgreSQL a ete prefere pour sa maturite et son support natif des types JSON, utilises pour stocker des listes et correspondances de configuration variables (langages cibles, correspondance de relecteurs) sans schema rigide.

5.3 Developpement des modules

5.3.1 Le moteur de revue

Le moteur de revue est organise en quatre paquets Python : `config` (chargement de `.aireviewer.yml` et modele `Settings`), `github` (authentification et acces a l'API GitHub), `llm` (client Azure OpenAI, analyseur parallele, prompts) et `models` (modeles Pydantic partages). L'annexe A presente un exemple complet de fichier de configuration.

5.3.2 Le tableau de bord

Le tableau de bord est structure en pages (`src/pages`), composants reutilisables (`src/components`) et une couche d'accès a l'API typee (`src/lib/api.ts`) qui centralise la gestion du jeton d'authentification et le renouvellement automatique de session en cas d'expiration.

5.4 Tests et debogage

5.4.1 Strategie de tests

La suite de tests unitaires s'exécute integralement hors ligne : tous les appels a GitHub et au fournisseur de LLM sont simules (*mock*), ce qui garantit des tests rapides, deterministes et ne consommant aucun quota d'API externe. Elle couvre la configuration, le client GitHub (authentification par jeton personnel *et* par application GitHub), la recuperation et la publication de PR, l'analyseur, le client LLM, l'orchestrateur (`Agent`) et le serveur webhook.

5.4.2 Resultats obtenus

Au moment de la redaction de ce rapport, la suite compte **92 tests**, tous passants. Le tableau 5.1 detaille la couverture de code par module du moteur de revue.

Module	Instructions	Couverture
<code>config/settings.py</code>	42	100 %
<code>core/agent.py</code>	24	100 %
<code>github/client.py</code>	111	95 %
<code>github/pr_fetcher.py</code>	11	100 %
<code>github/pr_publisher.py</code>	147	99 %
<code>llm/analyzer.py</code>	58	100 %
<code>llm/client.py</code>	18	100 %
<code>llm/prompts.py</code>	8	100 %
<code>models/</code> (pr, review)	98	100 %
Moteur de revue (total)	517	99 %
<code>webhook/app.py</code> (webhook + API du tableau de bord)	445	42 %

Tableau 5.1 – Couverture de tests du moteur de revue par module

Le moteur de revue — c’est-a-dire la partie deterministe et critique du systeme, responsable du calcul du score et du verdict — atteint ainsi une couverture de 99 %. La couche `webhook/app.py`, qui a fortement grossi avec l’ajout du tableau de bord (end-points CRUD, statistiques), reste en revanche partiellement couverte : la verification de signature et l’orchestration de la revue sont testees, mais la majorite des points d’accès `/api` ne le sont pas encore. Ce point est identifie comme axe d’amelioration prioritaire (voir la conclusion generale).

5.4.3 Processus de debogage

Le debogage s’est appuye sur l’execution ciblee de sous-ensembles de tests (`pytest -k`) et sur l’analyse de couverture ligne par ligne (`pytest -cov=prlens -cov-report=term-missing`) pour identifier les branches non exercees, notamment dans la gestion des cas d’echec partiel de l’analyseur.

5.5 Integration des composants

Le moteur de revue et le serveur webhook partagent un seul et meme code d’orchestration (`Agent.run`), ce qui garantit que les deux modes de deploiement produisent un resultat identique pour un meme contenu de Pull Request. Le tableau de bord s’integre au serveur webhook via une API REST authentifiee, sans dependance directe au code du moteur de revue.

5.6 Validation et verification

5.6.1 Evaluation par rapport aux besoins initiaux

Au-dela des tests unitaires — qui valident la logique deterministe du systeme — la pertinence des revues produites par le LLM a ete validee au moyen d’un **harnais d’évaluation** dedie (paquet `eval/`), constitue de quinze cas de diffs construits a la main avec un resultat attendu connu. Un cas est considere reussi si le systeme retourne un constat du type attendu, a un niveau de severite au moins egal a celui attendu (ou, pour les cas dits « propres », s’il ne retourne aucun commentaire).

Categorie	Cas	Reussis	Ce qui est verifie
Securite	5	5	Secrets codes en dur, injection SQL, traversee de repertoire, injection de commande
Qualite	4	4	Imbrication excessive, duplication, except nu, fonctions demesurees
Propre	3	2	Absence de faux positif sur du code effectivement correct
Mixte	3	2	Plusieurs types de problemes dans un meme diff (securite + qualite + performance)
Total	15	13	86,7 % de precision

Tableau 5.2 – Resultats du harnais d’évaluation (derniere execution)

Ce resultat depasse le seuil de precision minimal fixe pour le projet (70 %). Les deux echecs restants sont des faux positifs sur des cas « propres » limites, ou du code inhabituel mais correct declenche neanmoins un commentaire — une limite intrinseque et documentee de l’approche par LLM (voir conclusion generale).

5.6.2 Conformite aux specifications

La verification de conformite a porte sur chacune des exigences du backlog metier (tableau 2.2) : les deux modes de deploiement, la configuration par depot, l’authentification OAuth et la persistance de l’historique ont ete verifiees manuellement en conditions reelles, en plus des tests automatises.

5.7 Optimisation des performances

Le point de performance le plus sensible identifie est le temps de reponse de l’analyse d’une Pull Request touchant de nombreux fichiers. La parallelisation des appels au LLM

par un pool de travailleurs bornes (§4.6.2) constitue la principale optimisation mise en œuvre : elle transforme un temps d’analyse lineaire en nombre de fichiers en un temps proche de $\lceil n/\text{max_workers} \rceil$ appels sequentiels, tout en respectant les limites de debit du fournisseur de LLM.

5.8 Documentation technique

La documentation technique du projet comprend un fichier `README.md` exhaustif (installation des deux modes de deploiement, reference de configuration, reference des variables d’environnement, description de l’API REST du tableau de bord) ainsi que la documentation de conception dans le dossier `docs/`. Le present rapport en constitue la synthese academique.

5.9 Formation et transfert de competences

[A COMPLETER : Si applicable : preciser les modalites de transfert de competences (documentation utilisateur du tableau de bord, guide d’installation pour une equipe qui adopterait PRLens, etc.)]

Conclusion du chapitre

Ce chapitre a presente la realisation effective de PRLens et sa double validation : une couverture de tests unitaires proche de l’exhaustivite sur le moteur de revue, et un protocole d’évaluation quantitatif confirmant que la pertinence des revues produites depasse largement le seuil fixe pour le projet. Les limites identifiees — couverture partielle de la couche API du tableau de bord, faux positifs residuels sur du code atypique — sont reprises et discutees dans la conclusion generale.

Conclusion generale et perspectives

Ce projet visait a repondre a une problematique clairement identifiee dans le monde du developpement logiciel : la revue de code humaine, bien qu'indispensable, est un processus lent, couteux et inconstant, qui laisse regulierement passer des problemes evitables faute de temps ou d'attention disponible. PRLens y apporte une reponse concrete sous la forme d'un systeme de revue automatisee, capable d'analyser une Pull Request GitHub et de produire, en quelques dizaines de secondes et sans intervention humaine, des commentaires en ligne, un score de qualite et un verdict formel exploitables directement dans le flux de travail existant de l'equipe.

Le travail realise a couvert l'ensemble de la chaine, de l'analyse du besoin metier jusqu'a la validation quantitative des resultats produits : un moteur de revue partage entre deux modes de deploiement (pipeline sans etat ou application GitHub installee une fois pour tous les depots), un modele de scoring et de verdict deterministe construit sur une source non deterministe, et un tableau de bord web complet permettant a un utilisateur de gerer l'ensemble du systeme sans jamais toucher a un fichier de configuration. La validation a ete menee sur deux plans : une suite de 92 tests unitaires garantissant une couverture de 99 % du moteur de revue, et un harnais d'evaluation dedie confirmant une precision de 86,7 % des revues produites, nettement au-dessus du seuil de 70 % fixe pour le projet.

Ce travail n'est cependant pas exempt de limites, dont certaines sont intrinseques a l'approche retenue. Le LLM analyse chaque fichier isollement et n'a donc pas de visibilite sur les interactions entre fichiers d'une meme Pull Request ; seules les lignes presentes dans le diff sont examinees, les problemes preexistants dans le code non modifie restant hors perimetre par construction ; enfin, la non-determinisme propre aux LLM peut occasionnellement produire un faux positif sur du code inhabituel mais correct, comme l'a quantifie le harnais d'evaluation. Sur le plan de l'ingenierie logicielle, la couche API du tableau de bord — qui a grossi rapidement avec l'ajout des fonctionnalites de gestion des depots — reste en retrait sur la couverture de tests par rapport au moteur de revue.

Sur le plan personnel, ce projet a permis de mobiliser et de consolider des competences couvrant l'ensemble de la chaine de developpement d'une application moderne : conception d'une architecture logicielle partagee entre plusieurs points d'entree, integration avec des API tierces sensibles (GitHub, fournisseur de LLM) et leurs mecanismes d'authentification respectifs, modelisation de donnees relationnelles, developpement d'une interface web complete en React/TypeScript, et surtout mise en place d'un

protocole de validation rigoureux ne se limitant pas aux tests unitaires classiques mais mesurant également la qualite du resultat produit par un composant intrinsequement non deterministe.

Plusieurs perspectives d'evolution se degagent naturellement de ce bilan. A court terme, il conviendrait de renforcer la couverture de tests de la couche API du tableau de bord, en priorisant les points d'acces les plus sensibles (authentification, activation/desactivation d'un depot, deconnexion d'un depot) et de mettre en place un outil de migration de schema (Alembic) pour fiabiliser les evolutions futures de la base de donnees. A moyen terme, l'analyse pourrait s'etendre a un **contexte multi-fichiers**, en fournissant au LLM une vue partielle des fichiers lies au sein d'une meme Pull Request, afin de detecter des incoherences croisees. Enfin, a plus long terme, le projet pourrait s'ouvrir a d'autres fournisseurs de LLM au-dela d'Azure OpenAI, et enrichir le tableau de bord de fonctionnalites collaboratives (notifications, gestion d'equipes, tableaux de bord par organisation).

Bibliographie

- [1] RAMIREZ, Sebastian. *FastAPI Documentation*. [en ligne], [reference du 13 juillet 2026]. Disponible sur : <https://fastapi.tiangolo.com>
- [2] OPENAI. *OpenAI API Reference*. [en ligne], [reference du 13 juillet 2026]. Disponible sur : <https://platform.openai.com/docs>
- [3] MICROSOFT. *Azure OpenAI Service Documentation*. [en ligne], [reference du 13 juillet 2026]. Disponible sur : <https://learn.microsoft.com/azure/ai-services/openai>
- [4] GITHUB, INC. *GitHub REST API Documentation*. [en ligne], [reference du 13 juillet 2026]. Disponible sur : <https://docs.github.com/rest>
- [5] GITHUB, INC. *Building GitHub Apps*. [en ligne], [reference du 13 juillet 2026]. Disponible sur : <https://docs.github.com/apps>
- [6] PYGITHUB CONTRIBUTORS. *PyGithub Documentation*. [en ligne], [reference du 13 juillet 2026]. Disponible sur : <https://pygithub.readthedocs.io>
- [7] COLVIN, Samuel et al. *Pydantic v2 Documentation*. [en ligne], [reference du 13 juillet 2026]. Disponible sur : <https://docs.pydantic.dev>
- [8] BAYER, Michael. *SQLAlchemy Documentation*. [en ligne], [reference du 13 juillet 2026]. Disponible sur : <https://docs.sqlalchemy.org>
- [9] META PLATFORMS, INC. *React Documentation*. [en ligne], [reference du 13 juillet 2026]. Disponible sur : <https://react.dev>
- [10] OWASP FOUNDATION. *OWASP Top 10*. [en ligne], [reference du 13 juillet 2026]. Disponible sur : <https://owasp.org/www-project-top-ten>
- [11] HARDT, D. (ed.). *The OAuth 2.0 Authorization Framework*. RFC 6749, Internet Engineering Task Force, octobre 2012.
- [12] JONES, M., BRADLEY, J., SAKIMURA, N. *JSON Web Token (JWT)*. RFC 7519, Internet Engineering Task Force, mai 2015.

Appendices

ANNEXE A

Exemple de fichier de configuration `.aireviewer.yml`

```
1 llm_model: gpt-4o
2 min_severity: info
3
4 target_languages:
5   - python
6   - javascript
7   - typescript
8   - java
9
10 excluded_files:
11   - "*.lock"
12   - "*.min.js"
13   - "dist/**"
14   - "build/**"
15
16 reviewers_mapping:
17   security: "IsmailMechkene"
18   quality: "team:senior-devs"
19
20 max_workers: 5
21 approve_threshold: 80
22 changes_threshold: 50
23 large_pr_threshold: 20
```

Listing A.1 – Exemple de fichier de configuration place a la racine d'un depot

ANNEXE B

Exemple de commentaire genere par PRLens

```
1 Severite : critique
2
3 Cle API codee en dur dans le code source. Toute personne ayant
4 acces au depot peut lire ce secret, qui restera en outre present
5 dans l'historique Git meme apres suppression.
6
7 Suggestion : charger la cle depuis une variable d'environnement,
8 par exemple API_KEY = os.environ["OPENAI_API_KEY"].
```

Listing B.1 – Extrait d'un commentaire en ligne genere automatiquement

ANNEXE C

Extrait du modele de donnees (SQLAlchemy)

```
1 class Installation(Base):
2     __tablename__ = "installations"
3     __table_args__ = (
4         UniqueConstraint("user_id", "repo_name", name="uq_user_repo"),
5     )
6
7     id: Mapped[int] = mapped_column(primary_key=True)
8     user_id: Mapped[int] = mapped_column(ForeignKey("users.id"))
9     repo_name: Mapped[str] = mapped_column(String(150))
10    active: Mapped[bool] = mapped_column(Boolean, default=True)
11    connected: Mapped[bool] = mapped_column(Boolean, default=False)
12    min_severity: Mapped[str] = mapped_column(String(20))
13    approve_threshold: Mapped[int] = mapped_column(Integer)
14    changes_threshold: Mapped[int] = mapped_column(Integer)
```

Listing C.1 – Modele Installation (extrait simplifie)

ANNEXE D

Liste complete des variables d'environnement

Variable	Necessaire pour	Description
AZURE_OPENAI_API_KEY	Toujours	Cle d'API Azure OpenAI
AZURE_OPENAI_ENDPOINT	Toujours	Point de terminaison Azure OpenAI
GITHUB_TOKEN	Auth. par jeton	Jeton de repli en l'absence d'application GitHub
GITHUB_APP_ID	Auth. par App	Identifiant de l'application GitHub
GITHUB_APP_INSTALLATION_ID	Auth. par App	Identifiant d'installation de l'application
GITHUB_APP_PRIVATE_KEY_PATH	Auth. par App	Chemin local vers la cle private (.pem)
GITHUB_APP_PRIVATE_KEY_B64	Auth. par App	Cle private encodee en base64 (usage conteneur)
GITHUB_WEBHOOK_SECRET	Mode webhook	Secret de signature des livraisons webhook
DATABASE_URL	Tableau de bord	Chaine de connexion PostgreSQL
SESSION_SECRET	Tableau de bord	Cle de signature des jetons de session
GITHUB_OAUTH_CLIENT_ID	Tableau de bord	Identifiant client de l'application OAuth
GITHUB_OAUTH_CLIENT_SECRET	Tableau de bord	Secret client de l'application OAuth
FRONTEND_URL	Tableau de bord	Origine(s) autorisee(s) pour le CORS et la redirection post-OAuth
VITE_API_BASE_URL	Tableau de bord (client)	Origine du serveur API cote client

Tableau D.1 – Variables d'environnement du serveur PRLens

ANNEXE E

Liste complete des points d'accès de l'API REST

Methode	Chemin	Objet
GET	/	Vérification d'état du serveur
POST	/webhook	Reception des événements GitHub (signature HMAC vérifiée)
GET	/auth/github	Démarrage du flux OAuth
GET	/auth/callback	Retour OAuth, émission du jeton de session
GET	/api/user	Utilisateur connecté
GET	/api/stats	Statistiques du tableau de bord
GET	/api/repos	Dépôts connectés
GET	/api/repos/{name}	Détail d'un dépôt et de ses paramètres
GET	/api/github/repos	Dépôts GitHub disponibles à la connexion
POST	/api/repos/{name}/enable	Connexion d'un dépôt
DELETE	/api/repos/{name}	Déconnexion d'un dépôt (suppression de l'historique)
PUT	/api/repos/{name}/settings	Mise à jour des paramètres de revue d'un dépôt
PUT	/api/repos/{name}/active	Activation / désactivation de la revue d'un dépôt
GET	/api/reviews	Revue récentes, tous dépôts confondus
GET	/api/repos/{name}/reviews	Revue d'un dépôt donné

Tableau E.1 – Points d'accès exposés par le serveur FastAPI

Resume

Ce rapport presente PRLens, un systeme automatise de revue de code pour les Pull Requests GitHub, fonde sur un grand modele de langage (GPT-4o via Azure OpenAI). Le systeme analyse le diff de chaque Pull Request, detecte des problemes de securite, de qualite, de performance et de documentation, puis produit un score chiffre, un verdict formel et des commentaires en ligne publies automatiquement sur GitHub. Deux modes de deploiement sont proposes : un pipeline GitHub Actions sans serveur a heberger, et une application GitHub installee une fois pour tous les depots d'une organisation. Cette derniere est completee par un tableau de bord web (React/TypeScript) permettant la connexion des utilisateurs par OAuth GitHub, la configuration par depot et la consultation de l'historique des revues. La solution a ete validee par une suite de 92 tests unitaires (couverture de 99% du moteur de revue) et par un harnais d'evaluation quantitatif mesurant une precision de 86,7% des revues produites, superieure au seuil de 70% fixe pour le projet.

Mots-cles : revue de code, intelligence artificielle, grand modele de langage, GitHub, automatisaion, assurance qualite logicielle.

Abstract

This report presents PRLens, an automated code review system for GitHub Pull Requests, built on a large language model (GPT-4o via Azure OpenAI). The system analyzes the diff of every Pull Request, detects security, quality, performance, and documentation issues, and produces a numeric quality score, a formal review verdict, and inline comments published automatically on GitHub. Two deployment modes are offered : a server-less GitHub Actions pipeline, and a GitHub App installed once for every repository in an organization. The latter is complemented by a web dashboard (React/TypeScript) that lets users sign in with GitHub OAuth, configure review behaviour per repository, and browse recorded review history. The solution was validated through a suite of 92 unit tests (99% coverage of the review engine) and a quantitative evaluation harness measuring 86.7% precision on generated reviews, above the project's 70% target.

Keywords : code review, artificial intelligence, large language model, GitHub, automation, software quality assurance.